

Enhancing Performance and Maintainability in Unreal Engine 5 XR Applications through Optimization Techniques and Refactoring

Santos Rodriguez-Hughes¹, Terron Taylor²
Department of Computer Science, College of Liberal Arts and Sciences, Northern Illinois University

Introduction

The VR simulation "Vespa Mode" faced high latency and performance challenges common to Unreal Engine 5 (UE5) XR applications on standalone headsets. Unlike similar engines such as Unity, which excels in mobile VR, UE5 prioritizes high-quality rendering but struggles on limited hardware. UE5 VR devs need a plan if they want to break into this space.

Background

Berrezueta-Guzman & Wagner (2026) note Unity's lower hardware requirements make it "accessible to developers targeting devices with limited processing power... and projects with limited budgets."

Due to mobile-like hardware constraints, "around 70% of the games on [Meta Quest] are made by Unity" (Demant, 2023).

Despite UE5's advantages, Unity is still favored in the standalone XR space for its tutorials and accessible setup; UE5 often lacks comprehensive guides for the complex optimization process it requires.

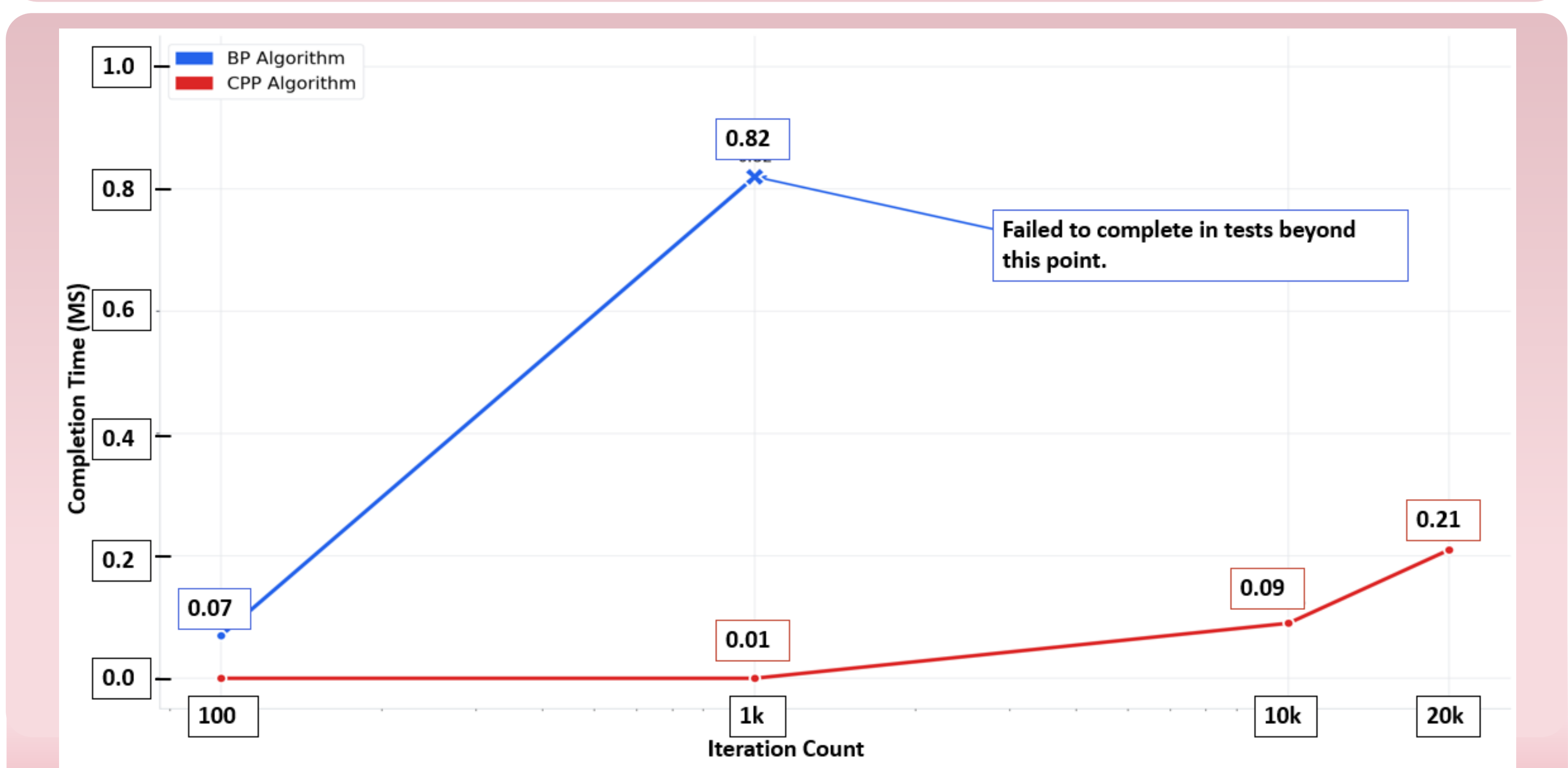
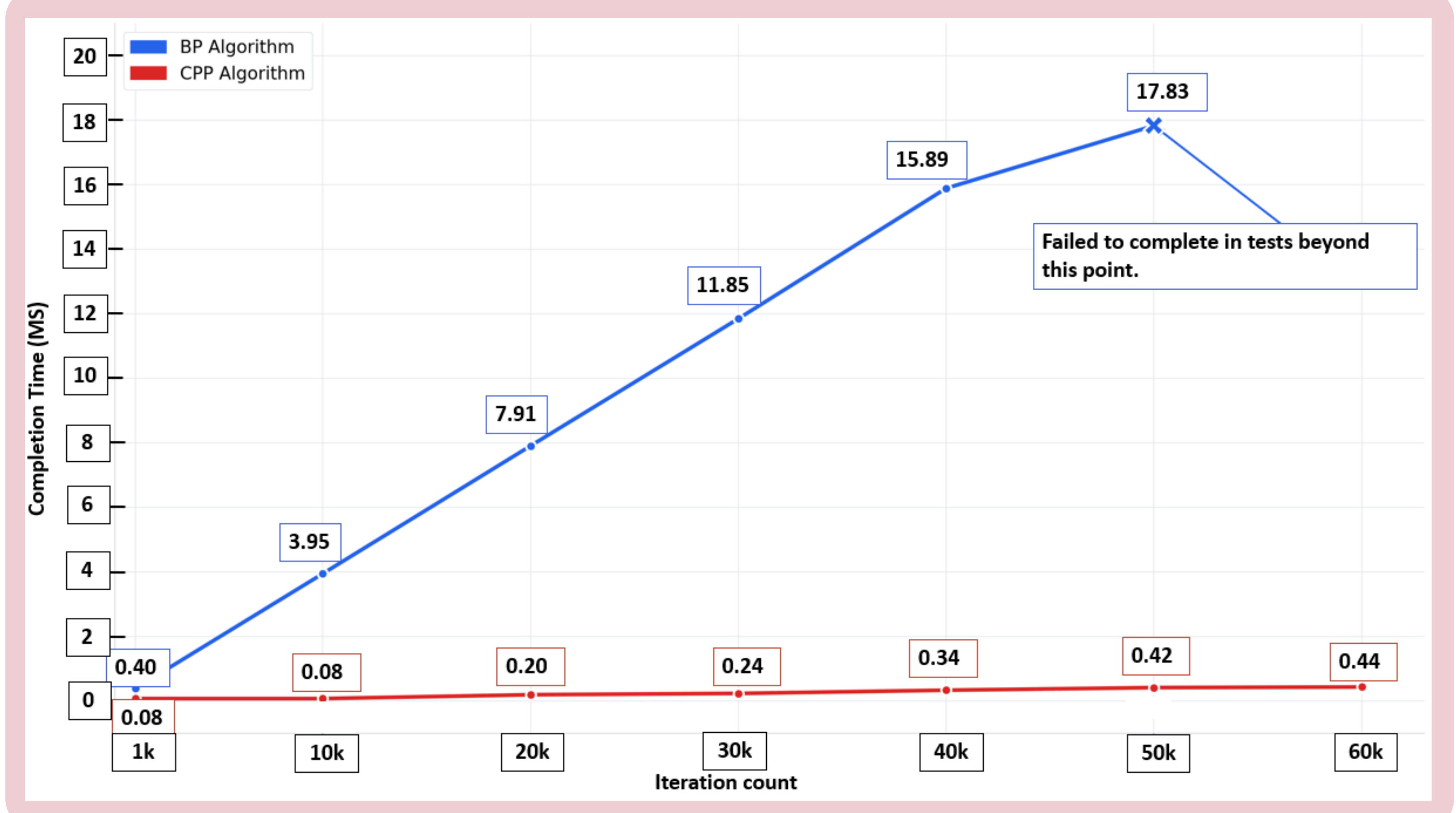
Research Goal

To create a replicatable process that applies optimization techniques in real-time rendering and C++ integration to improve performance and maintainability in UE5 location-based XR applications, enabling deployment on standalone XR hardware.

Methods

A strategy of targeted optimization was used, focusing on three key areas: Blueprint refactoring, level streaming, and water rendering.

- C++ Refactoring: Converted critical Blueprint functions on the Event Tick to C++
- HLODs: Used HLODs to merge distant objects into single proxy meshes
- Water: Replaced a heavy physics-based plugin with a custom, optimized material on a simple plane using panning UVs



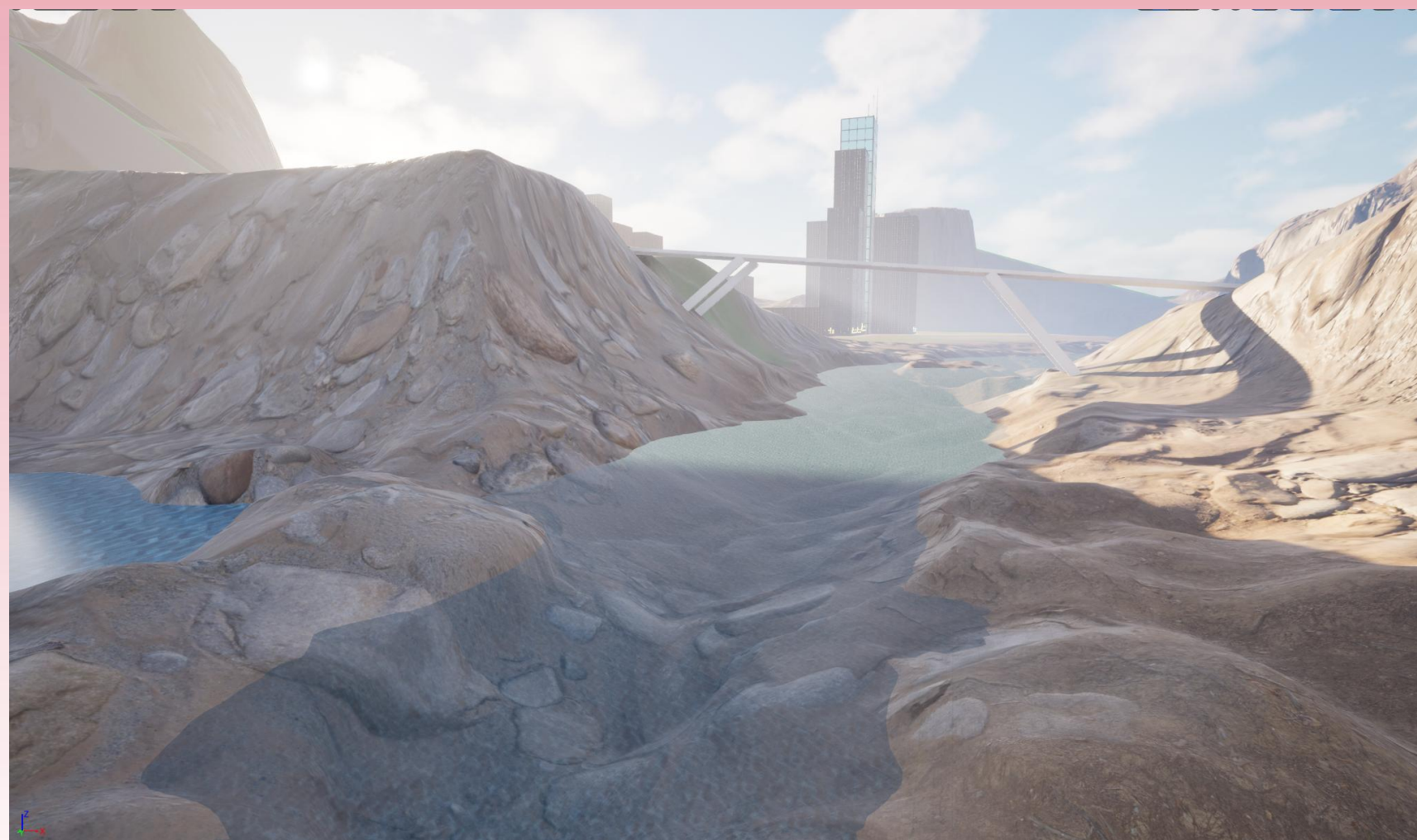
Isolated the execution time of identical logic with both systems

10 functions ran in a single frame; average logged per test

Second test performs maximum calculations possible on a tick in Vespa Mode



A side-by-side comparison of terrain in our simulation at different LOD levels			The left side: polygon complexity at close viewing distance			The right side: polygon complexity at far viewing distance		
Average	Max	Min	Average	Max	Min	Average	Max	Min
13,691,215.00	13,756,883.00	13,685,164.00	270,735.91	336,257.00	264,683.00			
13.7 million -> 336 thousand polygons post HLOD implementation (including static mesh + actor LOD reworks)								



- New water system reduced both polygon counts and draw calls
- Panned multiple tiling textures and adjusted opacity

Acknowledgements

Thanks to our mentor Federico Bassetti, 71 North Partnership Studio, Adam Bougher, Hannah Havel, OSEEL Undergrad research and Research Rookies

Conclusion

Results demonstrate a replicable framework for UE5 XR development, with future research extending to larger-scale XR systems, such as the NIU HuskieVerse initiative.

The optimization changes enabled successful deployment via ArborXR to a standalone headset, delivering a smooth and immersive user experience.

References

